perplexity

Step 6: Build your first regression suite

For your first hands-on project, use **Playwright with Python and Pytest** to test a public website. You will create a small regression suite, run it locally, and then run it automatically in GitHub Actions. [1] [2] [3]

Project goal

Test these critical flows on the Playwright website:

1. The home page loads and has the expected title.
2. The "Get started" link navigates to the documentation.
3. The Installation heading is visible after navigation.

These are simple, stable examples, but they demonstrate the same pattern used for real regression suites: open a page, perform an action, and assert the expected result. [3]

1. Set up the project

Install Python, then run:

```
mkdir my-regression-tests
cd my-regression-tests
python -m venv .venv
source .venv/bin/activate
pip install pytest pytest-playwright
playwright install
```

On Windows, use:

```
.venv\Scripts\activate
```

Pytest discovers files beginning with `test_` and test functions beginning with `test_`; this naming convention is required. [1] [3]

2. Create your first test file

Create a file named `test_regression.py`:

```
import re
from playwright.sync_api import Page, expect
```

```
BASE_URL = "https://playwright.dev/"

def test_home_page_title(page: Page):
    page.goto(BASE_URL)
    expect(page).to_have_title(re.compile("Playwright"))

def test_get_started_navigation(page: Page):
    page.goto(BASE_URL)
    page.get_by_role("link", name="Get started").click()
    expect(page.get_by_role("heading", name="Installation")).to_be_visible()
```

Each function is one regression test case. The first checks that the home page still loads correctly; the second checks that a core navigation journey still works. [3]

3. Run the suite locally

Run:

```
pytest
```

To run tests in Chromium only:

```
pytest --browser chromium
```

A passing result means the tested behavior still works. A failing result means either the application changed, the test expectation is outdated, or the test/environment is unstable. [1] [3]

4. Add a more realistic regression test

Create `test_cart_style_regression.py` to practice a user-journey test. This example uses Playwright's demo Todo app:

```
from playwright.sync_api import Page, expect

def test_add_todo_and_verify_count(page: Page):
    page.goto("https://demo.playwright.dev/todomvc")

    todo_box = page.get_by_placeholder("What needs to be done?")
    todo_box.fill("Learn regression testing")
    todo_box.press("Enter")

    expect(page.get_by_text("Learn regression testing")).to_be_visible()
    expect(page.get_by_test_id("todo-count")).to_contain_text("1")
```

This mirrors a real regression check: perform an action, verify the visible outcome, and repeat the same check after future changes. [3]

5. Run it in GitHub Actions

Create this file:

```
.github/workflows/playwright.yml
```

Add:

```yaml
name: Playwright Regression Tests

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.13'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest pytest-playwright

      - name: Install browsers
        run: python -m playwright install --with-deps

      - name: Run regression tests
        run: pytest --tracing=retain-on-failure

      - name: Upload failure traces
        if: ${{ !cancelled() }}
        uses: actions/upload-artifact@v4
        with:
          name: playwright-traces
          path: test-results/
```

This workflow checks out the code, installs Python and Playwright, runs the tests, and uploads diagnostic traces when a test fails. It runs on pushes and pull requests targeting the main branch.[2] [4]

6. Read the results

Result	Meaning	What to do
All tests pass	The checked behavior still works	Merge or continue development
One test fails consistently	Likely a real regression or outdated expectation	Reproduce it, inspect the trace, and fix the app or update the test
A test fails intermittently	Likely a flaky test	Improve waits, selectors, data isolation, or quarantine it for repair
Tests do not run	Environment or dependency issue	Check installation logs, browser installation, and workflow configuration [5] [6] [2]

If a test fails, Playwright's `--tracing=retain-on-failure` option saves a trace you can inspect to see the page state, actions, network requests, and console logs at the moment of failure.[4] [2]

Your next exercise

Expand the suite one test at a time:

- Add a test that opens the Playwright documentation and verifies the page title.
- Add a test that adds two Todo items and verifies the count becomes 2.
- Add a test that marks a Todo item complete and verifies it appears completed.
- Add one test for a deliberate failure, then use the trace to understand why it failed.

This gives you a practical foundation: identify important behavior, write repeatable assertions, automate them, run them on every change, and investigate failures systematically.[4] [1] [3]

⁂

1. https://docs.pytest.org/en/stable/getting-started.html
2. https://playwright.dev/python/docs/ci-intro
3. https://playwright.dev/python/docs/intro
4. https://playwright.dev/python/docs/ci
5. https://www.harness.io/blog/regression-testing-in-ci-cd-deliver-faster-without-the-fear
6. https://www.radview.com/blog/understanding-regression-testing-software/
7. https://playwright.dev/docs/ci-intro
8. https://playwright.dev/docs/ci
9. https://playwright.dev/java/docs/intro
10. https://www.learn.microsoft.com/en-us/training/modules/build-with-playwright/3-set-up-playwright/
11. https://learn.microsoft.com/bg-bg/azure/playwright-testing/quickstart-automate-end-to-end-testing
12. https://github.com/marketplace/actions/run-playwright-tests
13. https://github.com/microsoft/playwright-github-action
14. https://github.com/AutomationPanda/awesome-web-testing-playwright/blob/main/tutorial/07-github-actions.md

15. https://playwright.dev/java/docs/ci-intro

16. https://playwright.dev/dotnet/docs/ci-intro

17. https://playwright.dev/docs/intro